Rapport technique complet

SecuriGlass Factory OS

Architecture, implémentation, sécurité et vérification

Auteur	Mohamed Aziz Baoueb
Date	Octobre 2026
Nature	Documentation technique de preuve de concept (PoC)
Langages	Python, TypeScript, C, SQL, Bash
Référentiels	IEC 62443-3-3, OWASP ASVS v4.0, OWASP Top 10 for LLM, NIST SP 800-82, RGPD, EN 12150-1, ANSSI EBIOS RM

Ce document détaille l'implémentation technique du projet : pile logicielle, architecture, sécurité, tests et déploiement. Le rapport de projet, plus court, présente les objectifs et les fonctionnalités.

Table des matières

1 Introduction 3
1.1 Objet de ce document . . . 3
1.2 Principe de rédaction . . . 3

2 Architecture générale 4
2.1 Vue d'ensemble . . . 4
2.2 Le modèle de Purdue appliqué . . . 4

3 Pile technologique 5
3.1 Backend . . . 5
3.2 Frontend . . . 5
3.3 Automate et réseau industriel . . . 5
3.4 Infrastructure . . . 6

4 Structure du dépôt 7

5 Backend — API et services métier 8
5.1 Organisation du code . . . 8
5.2 Identité et authentification . . . 8
5.3 Autorisation : RBAC puis ABAC . . . 8
5.4 Confidentialité des données clients . . . 8
5.5 Journal d'audit chaîné . . . 8
5.6 Télémétrie en direct : WebSocket . . . 9
5.7 Détection d'anomalies (apprentissage automatique) . . . 9
5.8 Assistant technique et garde LLM . . . 9
5.9 API « usine » et principe du besoin d'en connaître . . . 10

6 Réseau industriel : automate et passerelle 11
6.1 Simulateur physique et serveur Modbus . . . 11
6.2 Passerelle de collecte . . . 11
6.3 Diode de données . . . 11

7 Interface web 12
7.1 Sécurité côté client . . . 12
7.2 Accès par rôle, dans l'interface comme sur le réseau . . . 12
7.3 Thème et langue . . . 12
7.4 Accessibilité et robustesse d'affichage . . . 12

8 Qualité, tests et intégration continue 13
8.1 Stratégie de tests . . . 13
8.2 Portes de qualité automatisées . . . 13
8.3 Vérification par attaque réelle . . . 13

9 Infrastructure et déploiement 14
9.1 Environnement de développement local 14
9.2 Durcissement système 14
9.3 Conteneurisation 14

10 Limites connues et travaux futurs 15
10.1 Ce qui est implémenté 15
10.2 Ce qui est volontairement hors périmètre 15
10.3 Suites possibles 15

A Glossaire 16

1 Introduction

1.1 Objet de ce document

Ce rapport documente, de façon exhaustive, l'implémentation technique de SecuriGlass Factory OS : la pile logicielle retenue pour chaque composant, la structure du dépôt, les mécanismes de sécurité réellement codés (et non seulement envisagés), la stratégie de tests, et la manière de faire tourner l'ensemble en local. Il complète le rapport de projet, qui reste centré sur les objectifs et les fonctionnalités vues par l'utilisateur.

1.2 Principe de rédaction

Chaque mesure de sécurité décrite ici a été vérifiée dans le code au moment de la rédaction. Quand une mesure n'est pas implémentée mais seulement prévue, elle est explicitement marquée *planifiée* ; ce choix évite qu'une lecture rapide du document fasse croire à une garantie qui n'existe pas encore dans le code.

2 Architecture générale

2.1 Vue d'ensemble

Le système est composé de cinq services indépendants, chacun dans son propre répertoire sous `services/`, communiquant par des interfaces strictement définies (Modbus TCP, UDP, HTTP, WebSocket). Aucun composant ne partage de mémoire ou de base de données avec un autre composant d'une zone de confiance différente.

Service	Rôle	Langage
`ot-plc-daemon`	Automate du four (simulateur + serveur Modbus)	C11
`ot-edge-gateway`	Passerelle de collecte OT → diode	Python 3.12
`backend-api`	API métier, authentification, IA, WebSocket	Python 3.12 / FastAPI
`frontend-ui`	Console web	TypeScript / React 18
`soc-siem`	Détection et corrélation (hors périmètre console)	Python

2.2 Le modèle de Purdue appliqué

Chaque service est rattaché à un niveau du modèle de Purdue, qui organise un système industriel en zones de confiance croissante :

Niveau	Zone	Composants et contraintes
0–1	Terrain / contrôle	`ot-plc-daemon` : aucune dépendance réseau sortante, aucune écriture distante non authentifiée.
2–3	Supervision OT	`ot-edge-gateway` : client Modbus en *lecture seule* uniquement (aucune fonction d'écriture implémentée côté client).
3,5	Zone démilitarisée	Diode de données : liaison UDP strictement unidirectionnelle, aucun accusé de réception possible par construction du protocole.
4	Informatique de gestion	`backend-api` + PostgreSQL : seul point d'entrée du système d'information vers les données OT.
5	Entreprise	`frontend-ui` : aucun accès réseau direct à l'OT, tout passe par l'API de niveau 4.

La propriété de sécurité centrale du système est que **le franchissement du niveau 3,5 vers le niveau 3 est impossible par construction** : la diode de données est un émetteur UDP *sans aucune fonction de réception*, donc même une compromission complète du niveau 4 ne donnerait pas, à elle seule, de chemin réseau vers l'automate.

3 Pile technologique

3.1 Backend

- **FastAPI** (Python 3.12, async) — framework HTTP et WebSocket.
- **asyncpg** — pilote PostgreSQL asynchrone, sans ORM, requêtes SQL paramétrées explicites.
- **Pydantic v2** — validation stricte des schémas d'entrée/sortie (`extra="forbid"`, rejet de tout champ inconnu).
- **Argon2id** (bibliothèque `argon2-cffi`) — hachage des mots de passe, paramètres $t = 3$, $m = 65536$ KiB (64 Mio), $p = 4$, sel de 16 octets, conformes à la RFC 9106.
- **PyJWT** — jetons de session signés HS256, identifiant unique (`jti`) permettant la révocation.
- **cryptography** (AES-256-GCM) — chiffrement au repos des données personnelles clients (TDE applicative).
- **scikit-learn** (repli pur Python si absent) — détection d'anomalies par forêt d'isolement.
- **Alembic** — migrations de schéma versionnées et idempotentes.
- **mypy –strict**, **bandit**, **black** — qualité et sécurité statique du code.

3.2 Frontend

- **React 18** + **TypeScript** (mode strict, `noUncheckedIndexedAccess`, `exactOptionalPropertyTypes`).
- **Vite** — empaquetage ; build de production sans script inline, compatible avec une Content-Security-Policy stricte.
- **Zustand** — état applicatif, sans middleware de persistance (le jeton de session ne vit qu'en mémoire JavaScript).
- **react-router** — routage côté client, gardes d'accès par rôle.
- **Vitest** + **Testing Library** — tests unitaires et de composants, couverture > 80 % imposée en seuil de build.
- Aucune bibliothèque de rendu HTML dynamique, aucune dépendance graphique tierce : les graphiques sont du SVG généré par le code.

3.3 Automate et réseau industriel

- **C11** (`-std=c11 -Wall -Wextra -Werror -pedantic`) — démon automate, zéro allocation dynamique résiduelle, `poll()` pour la boucle réseau non bloquante.
- **Modbus Application Protocol V1.1b3** — codes fonction 0x01 à 0x10 implémentés côté serveur (lecture de bobines, de registres, écriture simple et multiple).
- **Python 3.12** (passerelle) — client Modbus en lecture seule, `cryptography` pour la signature Ed25519 des trames.

3.4 Infrastructure

- **Docker** — conteneurisation de PostgreSQL (avec extension `pgAudit`) et des services applicatifs.
- **Ansible** — durcissement système conforme aux CIS Benchmarks (rôle `cis_linux`).
- **Terraform** — définition déclarative de l'infrastructure cible.
- **GitHub Actions** — intégration continue en sept étapes (chapitre 8).

4 Structure du dépôt

```
poc-glass/
  database/           Schemas SQL, migrations Alembic
    schemas/          01_init .. 06_factory_extension.sql
    migrations/versions/ 0001 .. 0004
  docs/               Architecture, GRC, specifications, QA
  infra/
    docker/           Dockerfile PostgreSQL + docker-compose
    ansible/          Durcissement CIS (rôle cis_linux)
    terraform/         Infrastructure déclarative
  redteam/            Outils de test d'intrusion Modbus (lecture
                       et écriture encadrées, autorisation requise)
  scripts/            dev_up.sh / dev_down.sh / seeds de démo
  security/           Config pipeline SAST/SCA, tests associés
  services/
    backend-api/      API FastAPI (app/api, app/core, app/services)
    frontend-ui/       Console React/TypeScript (src/)
    ot-plc-daemon/     Automate C11 (src/, include/, tests/)
    ot-edge-gateway/   Passerelle Python (gateway/)
    soc-siem/          Détection et corrélation
  tests/              Tests d'intégration et de sécurité transverses
```

5 Backend — API et services métier

5.1 Organisation du code

Le backend suit une séparation stricte en trois couches : `app/api` (routes HTTP/WebSocket, validation d'entrée, dépendances FastAPI), `app/services` (logique métier pure, testable indépendamment du framework web) et `app/core` (configuration, base de données, sécurité transverse).

5.2 Identité et authentification

Les mots de passe sont hachés en Argon2id (paramètres au chapitre 3). La connexion délivre un jeton JWT (HS256) contenant un identifiant unique de session (`jti`) ; ce `jti` est inscrit dans une table de révocation lors de la déconnexion, ce qui permet d'invalider un jeton avant son expiration naturelle sans avoir besoin d'une liste noire globale à chaque requête. Un verrouillage temporaire du compte suit plusieurs échecs de connexion consécutifs.

5.3 Autorisation : RBAC puis ABAC

Six rôles sont définis : `SUPER_ADMIN`, `SECURITY_AUDITOR`, `PLANT_MANAGER`, `OT_OPERATOR`, `QUALITY_INSPECTOR` et `SERVICE_ACCOUNT_EDGE`. Chaque route de l'API déclare explicitement les rôles autorisés via une dépendance FastAPI (`require_roles`), et toute tentative refusée est inscrite dans le journal d'audit avec le rôle détenu et le rôle requis. Un moteur de règles contextuelles (ABAC) vient compléter le RBAC pour des décisions qui dépendent du contexte (plage horaire, sous-réseau, verrou de maintenance) : la politique est *fail-closed* : en l'absence de donnée de contexte, la règle refuse plutôt que d'autoriser par défaut.

5.4 Confidentialité des données clients

Les champs personnels des clients (nom du contact, e-mail, téléphone) sont chiffrés avec AES-256-GCM avant écriture en base ; seule la raison sociale reste en clair, pour permettre la recherche sans exposer de donnée personnelle. Le déchiffrement n'a lieu qu'à la lecture par un rôle habilité.

5.5 Journal d'audit chaîné

Chaque événement sensible (connexion, création de client, refus d'accès, consigne bloquée, vérification de la chaîne elle-même) est inscrit dans `sec_data.audit_logs` avec une empreinte SHA-256 calculée sur l'enregistrement *et* l'empreinte de l'enregistrement précédent. Un verrou consultatif PostgreSQL (`pg_advisory_xact_lock`) sérialise les écritures concurrentes pour que la chaîne reste strictement ordonnée. Un déclencheur SQL interdit toute modification ou suppression de ligne existante `a posteriori` ; la suppression d'un compte (exigée par le RGPD, art. 17) ne touche donc jamais le journal, qui conserve un instantané de l'identité de l'acteur sans contrainte référentielle vers la table des utilisateurs.

5.6 Télémétrie en direct : WebSocket

Le canal `/ws/telemetry` authentifie le client dans le *premier message* après l'ouverture de la connexion (jamais dans l'URL, pour éviter qu'un jeton se retrouve dans des journaux d'accès). Le rôle est revérifié périodiquement (toutes les 30 secondes) pendant la durée de la connexion, et le nombre de connexions simultanées est plafonné. Chaque nouvelle mesure est accompagnée du diagnostic du modèle d'apprentissage automatique, jamais présentée comme « normale » par défaut quand ce diagnostic est indisponible.

5.7 Détection d'anomalies (apprentissage automatique)

Le détecteur utilise une forêt d'isolement (*Isolation Forest*, `scikit-learn`, avec un repli pur Python si la bibliothèque est absente) sur neuf mesures physiques (températures des quatre zones, pyromètre du verre, pression de trempe, vitesse de convoyeur, courant du soufflage, puissance consommée) enrichies de trois grandeurs dérivées : le régime de fonctionnement courant, l'écart maximal entre zones, et l'écart entre le pyromètre et la moyenne des zones. Ces grandeurs dérivées donnent au modèle le contexte croisé qu'une lecture zone par zone ne porte pas. En complément du score statistique, des signatures de pannes déterministes (surchauffe critique, perte de pression de trempe, blocage convoyeur, risque de casse thermique, défaillance d'une résistance) forcent une sévérité `CRITICAL` indépendamment du score, pour garantir leur détection à 100 % quelle que soit la sensibilité statistique du modèle.

5.8 Assistant technique et garde LLM

L'assistant est protégé par quatre barrières hermétiques, appliquées dans l'ordre :

1. **Filtrage de l'entrée** — canonicalisation du texte (décodage des échappements HTML/URL, normalisation Unicode NFKC, suppression des caractères de contrôle, mise à plat des variantes à base d'espacement ou de *leetspeak*, décodage récursif borné du Base64) avant comparaison à un ensemble de motifs d'attaque multilingues (français, anglais, espagnol) : contournement d'instruction, désactivation de sécurité, jeu de rôle, exfiltration de secret ou de message système.
2. **Recherche cloisonnée** — les réponses sont puisées dans un corpus documentaire local portant sur la norme EN 12150, jamais générées librement à partir de connaissances externes.
3. **Génération contrainte** — le moteur de réponse est déterministe et contraint au contexte normatif récupéré ; il n'improvise pas de valeur physique.
4. **Contrôle de sortie** — toute grandeur physique présente dans la réponse (température, pression, quelle que soit l'unité employée) est reconvertie et comparée aux bornes de sûreté EN 12150 (620–730°C, 1,5–6,5 bar) avant envoi ; une réponse hors bornes est bloquée et consignée comme incident critique.

Un limiteur de débit à seau de jetons (10 requêtes/minute/utilisateur) protège l'assistant contre un abus par volume. Les demandes portant sur des données financières ou personnelles des clients sont refusées par un filtre d'intention dédié, appliqué *après* le filtre anti-injection, et consignées séparément.

5.9 API « usine » et principe du besoin d'en connaître

L'endpoint `GET /api/v1/factory/overview` ne renvoie que les blocs de données utiles au rôle de l'appelant : un opérateur ne reçoit ni les données commerciales, ni la conformité, ni les alertes, même si le serveur les a calculées pour un autre appel. Ce filtrage est appliqué côté serveur, pas seulement masqué côté interface. La prise de commande (`POST /api/v1/orders`) exécute, dans une seule transaction : la vérification du stock disponible (verrouillage des lignes concernées), le calcul de la surface de verre, la déduction des matières premières et l'enregistrement de la commande ; tout échec annule l'ensemble.

6 Réseau industriel : automate et passerelle

6.1 Simulateur physique et serveur Modbus

Le démon `ot-plc-daemon` simule la thermodynamique d'un four à quatre zones de chauffe (inertie différentielle, refroidissement par soufflage, cinématique du convoyeur) et expose ces grandeurs via un serveur Modbus TCP non bloquant (`poll()`, un seul processus, aucune allocation dynamique résiduelle). Les consignes acceptées en écriture sont bornées physiquement dans le micrologiciel lui-même (ex. 500,0–750,0°C pour les consignes de zone) ; une valeur hors bornes est refusée par une exception Modbus (`ILLEGAL_DATA_VALUE`), indépendamment de tout contrôle applicatif en amont. L'arrêt d'urgence (E-STOP) coupe le chauffage immédiatement ; sa levée exige un acquittement physique du relais de sécurité et ne peut pas être demandée par le réseau seul, par analogie avec un automate réel.

6.2 Passerelle de collecte

La passerelle `ot-edge-gateway` interroge l'automate en Modbus avec un client strictement *read-only* (aucune fonction d'écriture n'est même implémentée côté client), décode les registres en grandeurs physiques, les valide (bornes physiques, absence de valeurs non finies), puis les transmet à la diode de données. Elle ne reçoit jamais de réponse du système d'information.

6.3 Diode de données

Le protocole binaire de la diode comprend un en-tête fixe de 64 octets (signature `SGDD`, version, numéro de séquence croissant sur 64 bits, horodatage en microsecondes, identifiant de l'émetteur, longueur de charge utile), suivi d'une charge utile JSON canonique et d'un sceau de 96 octets (empreinte SHA-256 et signature Ed25519). Le récepteur, côté système d'information :

- recalcule l'empreinte en temps constant (`hmac.compare_digest`) et vérifie la signature avec la clé publique de confiance de l'émetteur ;
- rejette toute trame dont la séquence n'est pas strictement croissante (protection anti-rejeu) et tolère une dérive d'horloge maximale de 30 secondes ;
- déduplique durablement par un identifiant de transmission unique (UUID v5) contraint `UNIQUE` en base, pour rejeter un rejeu même après un redémarrage de la passerelle ;
- journalise toute trame rejetée dans `ot_data.diode_import_log`, sans jamais émettre d'accusé de réception vers le réseau industriel.

7 Interface web

7.1 Sécurité côté client

La politique de sécurité du contenu (CSP) interdit tout script non auto-hébergé et tout script en ligne (`default-src 'none'`) ; le jeton de session ne vit qu'en mémoire JavaScript (aucune écriture dans `localStorage`, `sessionStorage` ou un cookie), ce qui est vérifié automatiquement par un test qui interdit la présence de ces appels dans l'ensemble du code source. Un rechargement de page déconnecte donc l'utilisateur par construction.

7.2 Accès par rôle, dans l'interface comme sur le réseau

La navigation, les routes et les sélecteurs d'état n'affichent une rubrique que si le rôle courant y donne droit ; ce filtrage côté client est une commodité d'usage, jamais une autorité : chaque appel réseau correspondant est revérifié côté serveur, et une tentative d'accès direct à une route non autorisée renvoie une page d'erreur sans qu'aucune donnée n'ait transité.

7.3 Thème et langue

L'interface suit la préférence système pour le mode clair ou sombre (`prefers-color-scheme`) et propose une bascule manuelle ; la console est disponible en français et en anglais via un dictionnaire de traduction central. Ces deux préférences restent en mémoire pour la durée de la session, sans être écrites sur le poste, pour respecter la même contrainte que le jeton de session.

7.4 Accessibilité et robustesse d'affichage

Les composants graphiques (courbes, jauges, carte réseau) sont du SVG généré par le code, jamais une image : aucune dépendance à un style en ligne interdit par la CSP. La mise en page reste lisible depuis un téléphone (375 px) jusqu'à un grand écran, avec un empilement explicite des éléments plutôt qu'une grille figée susceptible de faire chevaucher du texte long (ex. nom de client) sur une colonne voisine.

8 Qualité, tests et intégration continue

8.1 Stratégie de tests

Domaine	Outil	Résultat
API et services métier	pytest (avec PostgreSQL réel)	563 passés
Interface web	Vitest + Testing Library	122 passés
Automate C — unitaires	`bin/plc_test`	22 passés
Automate C — fuzzing	`bin/plc_fuzz_test`	763 cas, 0 crash
Automate C — fuites mémoire	`leaks` (macOS)	0 fuite
Passerelle de collecte	pytest	20 passés
Red team diode / LLM Guard / ML	pytest (adversarial)	133 passés

8.2 Portes de qualité automatisées

L'intégration continue (GitHub Actions) exécute sept étapes à chaque modification : `lint` (types stricts, formatage), `test` (suite complète), `sast` (analyse statique de sécurité, `bandit` et règles maison), `sca` (audit des dépendances), `docker` (construction des images), `sbom` (inventaire logiciel) et `sign` (signature des artefacts produits).

8.3 Vérification par attaque réelle

Au-delà des tests automatisés, le comportement du système a été vérifié par une attaque réseau réelle menée directement contre l'automate (sans passer par l'application), décrite au chapitre 6 du rapport de projet : elle a confirmé que les bornes physiques tiennent au niveau du micrologiciel, que le modèle de détection réagit sans intervention, et qu'elle a aussi révélé l'absence de sonde réseau sur le segment industriel — une limite honnêtement documentée plutôt que masquée.

9 Infrastructure et déploiement

9.1 Environnement de développement local

Le script `scripts/dev_up.sh` orchestre le démarrage complet de la chaîne en sept étapes : base PostgreSQL durcie (avec `pgAudit`), génération d'une paire de clés Ed25519 pour la passerelle, démarrage de l'API, création des comptes et données de démonstration, compilation et démarrage de l'automate, démarrage du récepteur et de la passerelle, puis démarrage de la console. `scripts/dev_down.sh` arrête proprement l'ensemble.

9.2 Durcissement système

Le rôle Ansible `cis_linux` applique un sous-ensemble des CIS Benchmarks : durcissement `sysctl` (désactivation du *forwarding* IP, protection anti-usurpation, activation des *SYN cookies*, désactivation des redirections ICMP), configuration de `sshd` et d'`auditd`, bannière légale de connexion.

9.3 Conteneurisation

PostgreSQL tourne dans une image Debian (et non Alpine, qui ne fournit pas l'extension `pgAudit`) avec principe de moindre privilège (`no-new-privileges`, capacités Linux réduites au strict nécessaire).

10 Limites connues et travaux futurs

10.1 Ce qui est implémenté

Tout ce qui est décrit dans ce rapport correspond à du code existant et testé au moment de la rédaction.

10.2 Ce qui est volontairement hors périmètre

- **Détection réseau sur le segment OT** — une attaque Modbus directe sur l'automate n'est pas aujourd'hui détectée par le flux d'alertes de la console (voir chapitre 6 du rapport de projet).
- **Réplication à froid et plan de reprise** — cités dans le dossier d'architecture comme mesures cibles, non implémentés dans ce PoC.
- **Signature des consignes envoyées à l'automate** — l'écriture de consignes existe au niveau du protocole Modbus pour les besoins de la démonstration, sans couche de signature applicative supplémentaire.
- **Image Docker *distroless*** pour le backend — non construite dans ce PoC.
- **Homologation** — ce projet est une démonstration technique ; aucune autorité d'homologation ne s'est prononcée sur le système.

10.3 Suites possibles

Ajouter une sonde de détection sur le réseau industriel, exécuter l'intégralité de la suite de tests (y compris ceux aujourd'hui ignorés faute d'outillage disponible sur macOS) sur un environnement Linux de référence, puis traiter dans l'ordre les mesures listées ci-dessus comme *planifiées*.

A Glossaire

ABAC *Attribute-Based Access Control* — autorisation fondée sur des règles contextuelles, en complément du contrôle par rôle.

CSP *Content Security Policy* — politique de sécurité du contenu, appliquée par le navigateur.

Diode de données Dispositif garantissant qu'un flux d'information ne peut transiter que dans un seul sens entre deux zones réseau.

IEC 62443 Famille de normes de cybersécurité pour les systèmes d'automatisation et de contrôle industriels.

Modèle de Purdue Modèle de référence organisant un système industriel en niveaux de confiance, du terrain (niveau 0) à l'entreprise (niveau 5).

Modbus TCP Protocole de communication industrielle utilisé pour interroger et commander un automate.

OT / IT *Operational Technology* (réseau industriel) et *Information Technology* (réseau informatique de gestion).

RBAC *Role-Based Access Control* — autorisation fondée sur le rôle de l'utilisateur.

TDE *Transparent Data Encryption* — chiffrement des données au repos, transparent pour l'application qui les consomme.